SALPHOENIX®1A 系列 FPGA DSP 用户手册

上海安路信息科技股份有限公司

UG_904（v1.1）2022 年 06 月

Confidential

目 录

目 录 I

1 概述 1

1.1 背景介绍 1

1.2 功能概述 1

2 DSP 功能 1

2.1 DSP 结构 1

2.2 DSP 特性描述 2

2.3 SIOU 单元 3

2.3.1 MREG 子单元 3

2.3.2 REG 子单元 4

2.4 AU 单元 5

2.5 MU 单元 6

2.6 TU 单元 7

2.7 LU 单元 9

3 功能组合拼接应用场景 11

4 用户使用模式 11

4.1 HDL 行为级描述 11

4.2 HDL 结构级描述 12

5 DSP 应用示例 13

5.1 乘法 13

5.2 加法/减法 13

5.3 累加 14
5.4 乘加 15
5.5 乘累加 16
5.6 带预加的乘法 16
5.7 带预加的乘累加 17
5.8 数据动态切换 18
5.9 级联实现宽位宽加法器 20
5.10 级联实现宽位宽乘法器 21
5.11 按位逻辑操作 21
6 附录 1-组合拼接应用情景示例代码 22
7 版本信息 30
免责声明 30

1 概述

1.1 背景介绍

数字信号处理中将使用大量的加法、乘法和累加运算；在 FPGA 中如利用 LUT 实现，较大的延迟将限制处理速率；利用 PH1A 系列 DSP 实现以上操作，将大大的降低处理延迟。

1.2 功能概述

SALPHOENIX®1A（以下简称 PH1A）系列 FPGA 的 DSP 采用微型总线式架构，包含五大处理单元：包括两输入加法器、乘法器、三输入加法器、逻辑运算单元、SIOU。

- 支持两输入的 27-bit 加法运算；
- 支持三输入带进位 54-bit 加法运算；
- 支持将三输入带进位 54-bit 加法器拆分为两个 27-bit 加法器；
- 支持进行输入数 EXTEND、ROUND、SHIFT，输入进位取反；
- 支持 OVERFLOW 和 UNDERFLOW；
- 支持 27 x 18-bit 乘法器（有符号的二进制补码）；
- 支持两输入 54-bit 按位逻辑操作，包括 AND、NAND 、OR、NOR、XOR、XNOR 和 NOT；
- 支持宽位异或操作，单个 DSP 可实现最多 108-bit XOR 操作，或拆分为两个 54-bit XOR 操作；
- 支持对输入信号进行一级或两级锁存；
- 支持对输入信号进行动态选择切换；
- 支持快速输入输出互联。

2 DSP 功能

2.1 DSP 结构

DSP 整体结构是微型总线式架构，如图 2-1 所示，包含一个 27-bit 的两数加法器 ADD UNIT （AU），一个带进位的 54bit 三数加法器 TADD UNIT （TU），一个 27 x 18bit 的乘法器 MULT UNIT （MU），一个 54-bit 逻辑单元 LOGIC UNIT （LU），和提供输入输出信号的总线单元 SIOU。AU 是 27-bit 的加法器。TU 是 54-bit 的三数加法法器。MU 是 27x18-bit 的有符号乘法器。LU 是 54-bit 的逻辑操作单元。SIOU 单元是 DSP 模块的控制核心，提供运算和逻辑单元的所有控制、输入、输出信号，包括信号的锁存、直通、切换，这种总线式架构为用户提供了灵活可配置的多种运算和逻辑组合功能。

图 2-1 输入信号：CLK, RST, CE, OPCTRL。左边输入信号：A, B, Y, Z, PI, CI。右边输出信号：

AO, BO, YO, PO, CO, WXOR, OVERFLOW, UNDERFLOW。

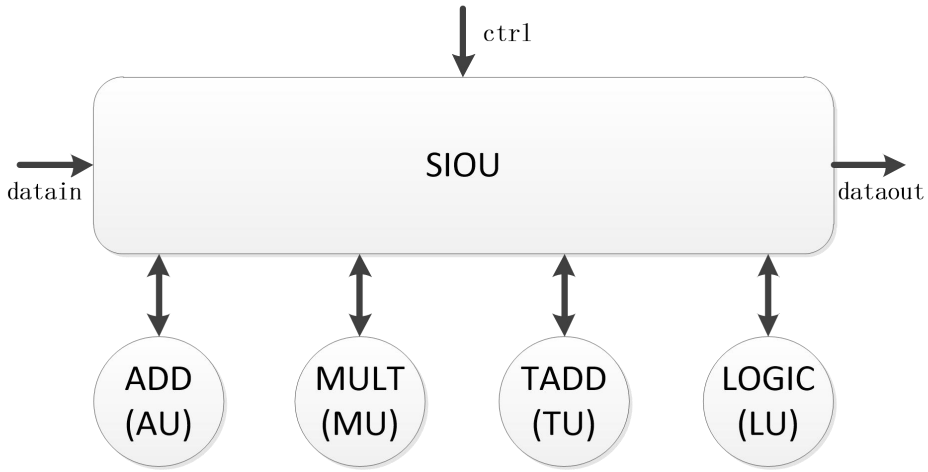

图 2-1 DSP 简化结构

2.2 DSP 特性描述

微型总线式 DSP 架构用户提供了灵活可配置的多种运算和逻辑组合功能。用户可以直接使用 AU、TU、MU、LU 的功能或者灵活拼接的组合功能。

AU：

- 支持两输入的 27-bit 加法运算；

TU：

- 支持三输入带进位 54-bit 加法运算；
- 支持将三输入加法器拆分为两个 27-bit 加法器；
- 支持进行输入数 EXTEND、ROUND、SHIFT，输入进位取反；
- 支持 OVERFLOW 和 UNDERFLOW；

MU：

- 支持 27 x 18-bit 乘法器（有符号的二进制补码）；

LU:

- 支持两输入 54-bit 按位逻辑操作，包括 AND、NAND 、OR、NOR、XOR、XNOR 和 NOT；
- 支持宽位异或操作，单个 DSP 可实现最多 108-bit XOR 操作，或拆分为两个 54-bit XOR 操作。

SIOU：

- 支持对输入信号进行一级或两级锁存；
- 支持对输入信号进行动态选择切换；
- 支持快速输入输出互联。

TD 软件通过对用户开放 PH1_PHY_DSPADD、PH1_PHY_DSPTADD、PH1_PHY_DSPMULT、PH1_PHY_DSPLOGIC、PH1_PHY_DSPREG、PH1_PHY_DSPMREG 六个基本源语来实现 DSP 的丰富功能。用户通过界面 IP Generator 生成设定的 DSP 功能的.v 文件，或用行为级描述编程、由 TD 软件自动推断，当然也可以直接在 HDL 里例化源语来实现的 DSP 功能。

表 2-1 DSP 功能单元和源语的对照表

功能单元	对应的源语
AU	PH1_PHY_DSPADD
TU	PH1_PHY_DSPTADD
MU	PH1_PHY_DSPMULT
LU	PH1_PHY_DSPLOGIC
SIOU	PH1_PHY_DSPREG，PH1_PHY_DSPMREG

2.3 SIOU 单元

SIOU 单元是 DSP 模块的控制核心，提供运算和逻辑单元的所有控制、输入、输出信号，包括信号的锁存、直通、切换，这种总线式架构为用户提供了灵活可配置的多种运算和逻辑组合功能。

用户用 PH1_PHY_DSPREG 和 PH1_PHY_DSPMREG 来实现 SIOU 功能。

2.3.1 MREG 子单元

Symbol 模型如下图所示。

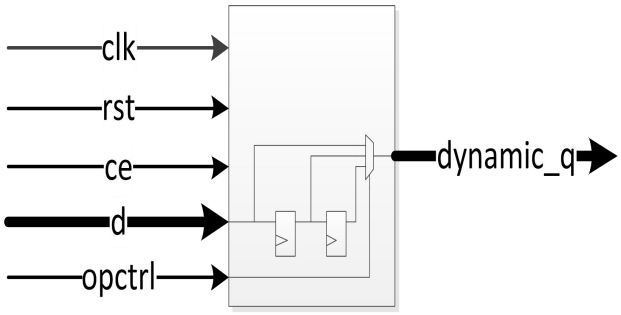

图 2-2 MREG Symbol 模型

表 2-2 MREG 端口描述

引脚名称	输入/输出	位宽/bit	描述
clk	input	1	时钟信号
rst	input	1	复位信号，可选异步复位和同步复位
ce	input	1	时钟使能信号
d	input	位宽可配	数据输入端口
opctrl	input	1bit	控制信号端口 0：dynamic_q 为 d 端 2 级寄存器输出 1：dynamic_q 为 d 端 0 级或 1 级寄存器输出 DYNAMIC_DATA = “D”：dynamic_q 为 d 端 0 级寄存。 DYNAMIC_DATA = “Q”：dynamic_q 为 d 端 1 级寄存。
dynamic_q	output	位宽可配	数据动态输出端口

表 2-3 MREG 参数配置

Attribute	Allowed Values	Default	Descriptions
WIDTH	1~54	1	数据 d、dynamic_q 的位宽
RSTMODE	" SYNC "," ASYNC "	" ASYNC "	寄存器复位方式，同步复位或异步复位
CLKMUX	" 0 "," 1 "" SIG "," INV "	"SIG "	时钟选择信号
CEMUX	" 0 "," 1 "" SIG "," INV "	" 1 "	时钟使能信号
RSTMUX	" 0 "," 1 "" SIG "," INV "	" 0 "	复位选择信号
DYNAMIC_DATA	"D","Q"	" Q "	配置 dynamic_q 寄存器级数 (0-2)

2.3.2 REG 子单元

Symbol 模型：

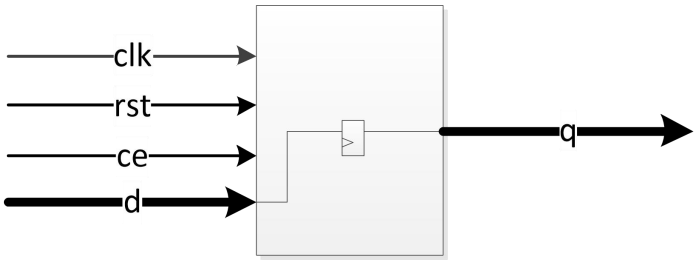

图 2-3 REG Symbol 模型

表 2-4 REG 端口描述

引脚名称	输入/输出	位宽/bit	描述
clk	input	1	时钟信号
rst	input	1	复位信号，可选异步复位和同步复位
ce	input	1	时钟使能信号
d	input	位宽可配	数据输入端口
q	ouput	位宽可配	数据 d 寄存 1 级输出

表 2-5 REG 参数配置

Attribute	Allowed Values	Default	Descriptions
WIDTH	1~54	1	数据 d、q 的位宽
RSTMODE	" SYNC "," ASYNC "	"ASYNC "	寄存器复位方式，同步复位或异步复位
CLKMUX	" 0 "," 1 "" SIG "," INV "	"SIG "	时钟选择信号
CEMUX	" 0 "," 1 "" SIG "," INV "	" 1 "	时钟使能信号
RSTMUX	" 0 "," 1 "" SIG "," INV "	" 0 "	复位选择信号

2.4 AU 单元

Symbol 模型如下图所示，AU 单元端口描述如表 2-6。

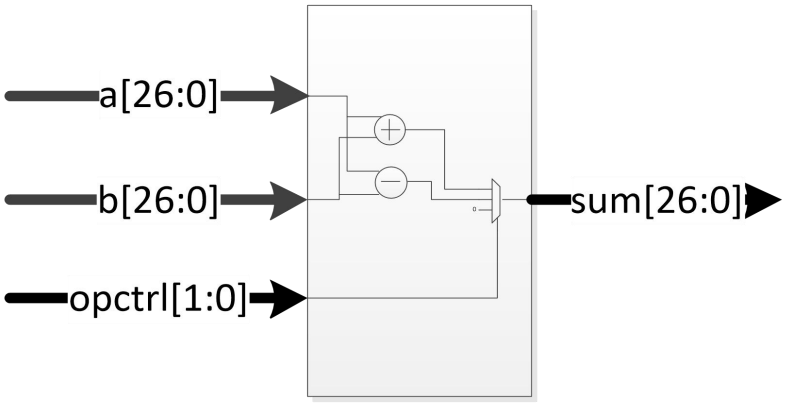

图 2-4 AU Symbol 模型

表 2-6 AU 端口描述

引脚名称	输入/输出	位宽/bit	描述
a	input	27	加法器数据输入端口 a
b	input	27	加法器数据输入端口 b
opctrl	input	2	加法器控制输入端口 opctrl[0]：加法器使能 opctrl[1]:0：执行 sum=a+b　1：执行 sum=a-b
sum	ouput	27	加法器输出端口

2.5 MU 单元

Symbol 模型如下图所示。

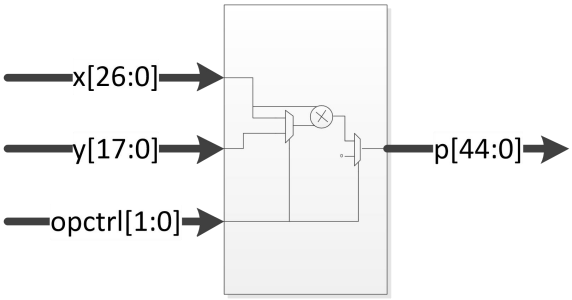

图 2-5 MU Symbol 模型

表 2-7 MU 端口描述

引脚名称	输入/输出	位宽/bit	描述
x	input	27	乘法器数据输入端口 x
y	input	18	乘法器数据输入端口 y
opctrl	input	2	加法器控制输入端口 opctrl[0]：控制乘法器使能 opctrl[1]：0：执行 p=x[26:0]*x[17:0] 1：执行 p=x*y
p	ouput	45	乘法器输出端口

2.6 TU 单元

TU 单元的输入端口可以动态选择，方便用户进行动态表达式计算。Symbol 模型如下图所示。

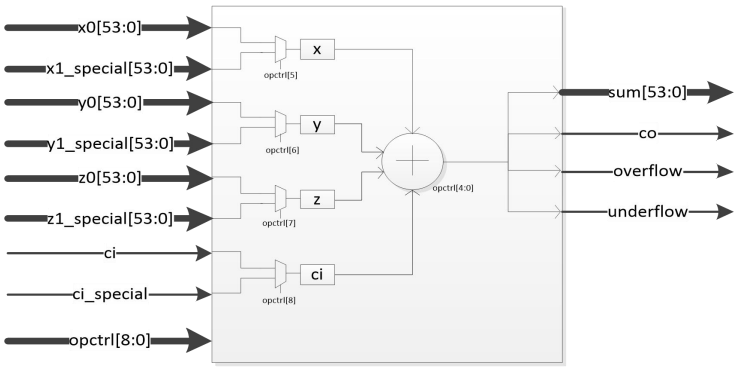

图 2-6 TU Symbol 模型

表 2-8 TU 端口描述

引脚名称	输入/输出	位宽/bit	描述
x0	input	54	加法器数据输入，支持 x0 和 x1_special 动态切换
y0	input	54	加法器数据输入，支持 y0 和 y1_special 动态切换
z0	input	54	加法器数据输入，支持 z0 和 z1_special 动态切换
ci	input	1	加法器进位输入，支持 ci 和 ci_special 切换
x1_special	input	54	加法器数据输入，支持 x0 和 x1_special 动态切换
y1_special	input	54	加法器数据输入，支持 y0 和 y1_special 动态切换
z1_special	input	54	加法器数据输入，支持 z0 和 z1_special 动态切换
ci_special	input	1	加法器进位输入，支持 ci 和 ci_special 切换
opctrl	input	9	加法器模式控制信号输入
sum	input	54	加法器数据输出
co	ouput	1	加法器进位输出，TU 的 co 和 ci 为快速进位信号，只能从一个 TU 到另一个 TU。
overflow	ouput	1	加法结果向上溢出
underflow	ouput	1	加法结果向下溢出

表 2-9 TU 参数配置

Attribute	Allowed Values	Default	Descriptions
X1_EXTEND	"YES "," NO "	" NO "	YES：对 x1_special 进行符号位扩展，{{9{x1_special[44]}},x1_special[44:0]。 NO：不改变 x1_special 的值。
Y1_ROUND	"YES "," NO "	" NO "	YES：把 y1_special 的值改变为常数 RND_CONST。 NO：不改变 y1_special 的值。
Z1_SHIFT	"YES "," NO "	" NO "	YES：对 z1_special 进行右移 17 位，高位填充符号位。 NO：不改变 z1_special 的值。
CI_INVERT	"YES "," NO "	" NO "	YES：对 ci_special 进行取反。 NO：不改变 ci_special 的值。
RND_CONST	0~0x3FFFFFFFFFFFFF	54’b0	DSP 模块内置常数。
USE_OVERFLOW	" S0" ~" S53"	" S53 "	sum 的溢出判断，溢出检测位置为 0-53

表 2-10 TU 控制信号 opctrl[8:4]

Attribute	Descriptions
opctrl[8]	ci 输入动态切换： 0：ci 1：ci_special
opctrl[7]	z 输入动态切换： 0：z0 1：z1_special
opctrl[6]	y 输入动态切换： 0：y0 1：y1_special
opctrl[5]	x 输入动态切换： 0：x0 1：x1_special
opctrl[4]	0：使能 co 1：使能 overflow 和 underflow

表 2-11 TU 控制信号 opctrl[3:0]

Attribute				Descriptions
opctrl[3]	opctrl[2]	opctrl[1]	opctrl[0]	sum
0	0	0	0	sum=x+y+z+ci
0	0	0	1	sum[26:0]=x[26:0]+y[26:0]+z[26:0] sum[53:27]=x[53:27]+y[53:27]+z[53:27]
0	0	1	0	sum=-x+y+z+ci
0	0	1	1	sum[26:0]=-x[26:0]+y[26:0]+z[26:0] sum[53:27]=x[53:27]+y[53:27]+z[53:27]
0	1	0	0	sum=x-y+z+ci
0	1	0	1	sum[26:0]=x[26:0]-y[26:0]+z[26:0] sum[53:27]=x[53:27]-y[53:27]+z[53:27]
0	1	1	0	sum=-x-y+z

Attribute				Descriptions
opctrl[3]	opctrl[2]	opctrl[1]	opctrl[0]	sum
0	1	1	1	sum[26:0]=-x[26:0]-y[26:0]+z[26:0] sum[53:27]=-x[53:27]-y[53:27]+z[53:27]
1	0	0	0	sum=x+y-z+ci
1	0	0	1	sum[26:0]=x[26:0]+y[26:0]-z[26:0] sum[53:27]=x[53:27]+y[53:27]-z[53:27]
1	0	1	0	sum=-x+y-z
1	0	1	1	sum[26:0]=-x[26:0]+y[26:0]-z[26:0] sum[53:27]=-x[53:27]+y[53:27]-z[53:27]
1	1	0	0	sum=x-y-z
1	1	0	1	sum[26:0]=x[26:0]-y[26:0]-z[26:0] sum[53:27]=x[53:27]-y[53:27]-z[53:27]

2.7 LU 单元

Symbol 模型：

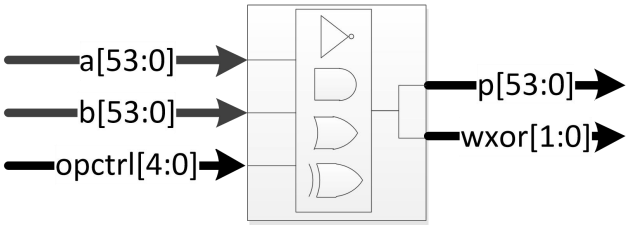

图 2-7 LU Symbol 模型

表 2-12 LU 端口描述

引脚名称	输入/输出	位宽/bit	描述
a	input	54	逻辑运算输入端口 a
b	input	54	逻辑运算输入端口 b
opctrl	input	5	逻辑运算控制信号
p	output	54	逻辑运算输出
wxor	output	2	宽位异或输出

表 2-13 LU 控制信号 opctrl[4]

Attribute	Descriptions
opctrl[3:0]	0
opctrl[4]	0：wxor[0] = 0；wxor[1] = ^(a[53:0]^b[53:0])； 1：wxor[0] =^(a[26:0] ^ b[26:0])；wxor[1] = ^(a[53:27] ^

	b[53:27])

表 2-14 LU 控制信号 opctrl[3:0]

Attribute				Descriptions
opctrl[3]	opctrl[2]	opctrl[1]	opctrl[0]	p
0	0	0	0	a xor b
0	0	1	0	a xnor b
0	0	0	1	a and b
0	0	1	1	(not a) and b
1	0	0	1	a and (not b)
1	0	1	1	a nor b
0	1	0	1	a or b
0	1	1	1	(not a) or b
1	1	0	1	a or (not b)
1	1	1	1	a nand b

3 功能组合拼接应用场景

PH1A DSP 支持多种运算和逻辑功能的丰富组合来实现很多功能，如乘加、加乘、加乘加等。加乘加组合的原语模块拼接示例如图 3-1 所示。

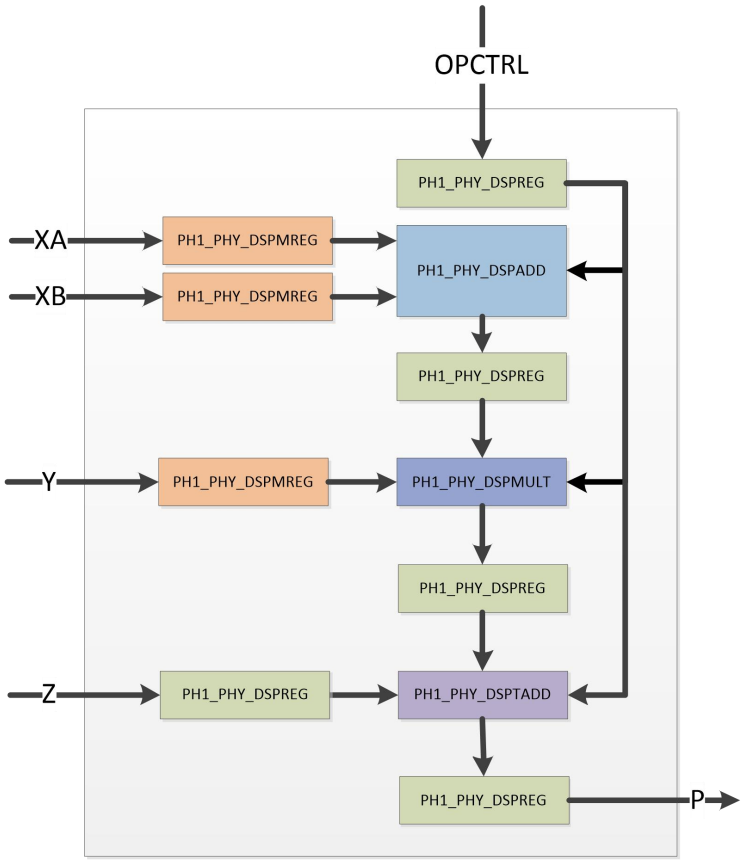

图 3-1 模块连接示例

4 用户使用模式

4.1 HDL 行为级描述

用户用 HDL 语言进行行为级描述，软件自动推断。

乘法代码示例：

```
always@(posedge clk)
begin
    out <= a*b;
```

```
end
```

预加乘代码示例：

```
always@(posedge clk)
begin
    out <= (a±b)*c;
end
```

累加乘代码示例：

```
always@(posedge clk)
begin
    out <= out± a * b;
end
```

4.2 HDL 结构级描述

用户用 HDL 语言进行结构级描述，软件遵循用户的结构化描述。用户用 DSP primitive 组合需要的运算功能，软件自动映射到芯片上的物理功能单元和互联资源。

用户可用的 DSP 源语有 6 个 primitive：PH1_PHY_DSPADD、PH1_PHY_DSPLOGIC、PH1_PHY_DSPMREG、PH1_PHY_DSPMULT、PH1_PHY_DSPREG、PH1_PHY_DSPTADD 详情见第 2 章描述。附录 1 展示了 6 个 dsp primitive 描述的复合计算单元，端口连接后面的注释表示可选连接信号。

5 DSP 应用示例

5.1 乘法

DSP 可实现 27 x 18 的乘法器，如 P=X*Y，输入输出的寄存器都是可选的。如图 5-1 所示：

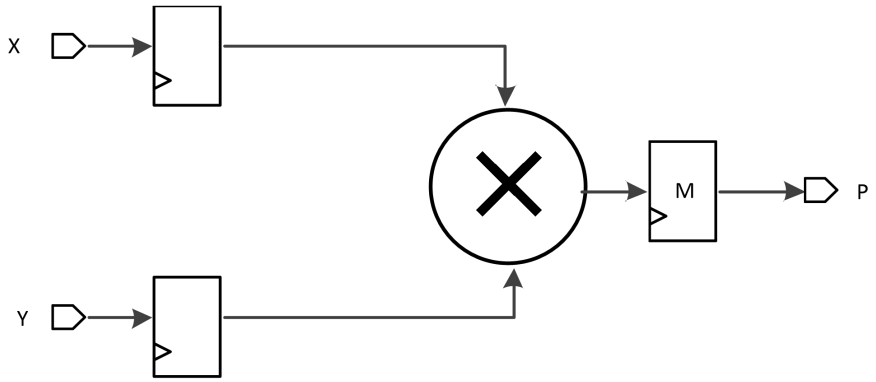

图 5-1 DSP 乘法操作简图

使用 DSP 子单元搭建乘法器，结构图如图 5-2 所示：

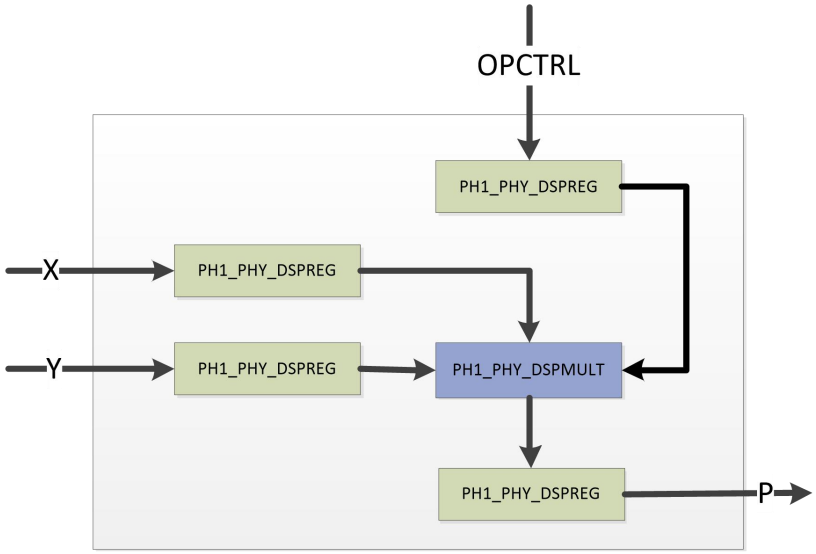

图 5-2 DSP 乘法操作结构图

5.2 加法/减法

DSP 实现 54-bit 加法或减法器，P=X + Z

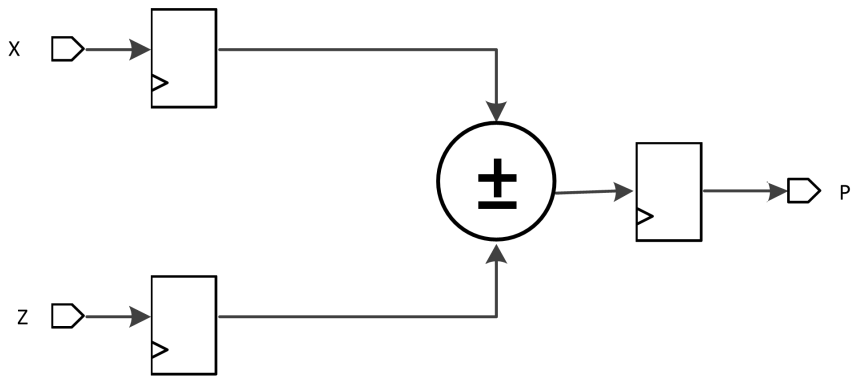

图 5-3 DSP 加法操作简图

使用 DSP 子单元搭建 54-bit 加法或减法器，结构图如图 5-4 所示：

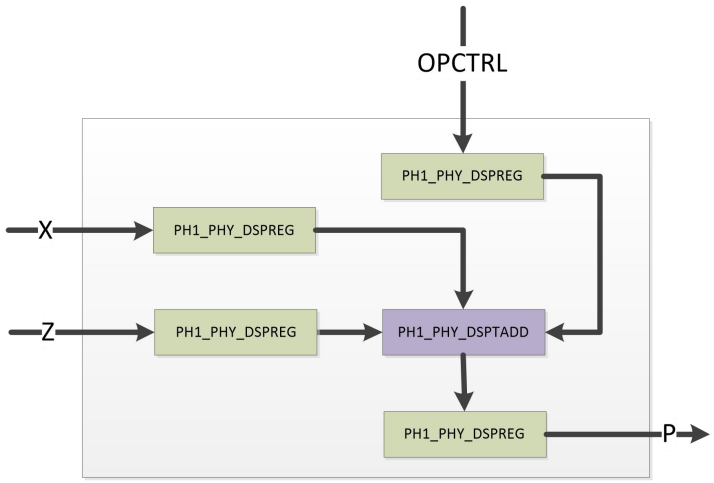

图 5-4 DSP 加法操作结构图

5.3 累加

DSP 实现累加功能：P=P+Z

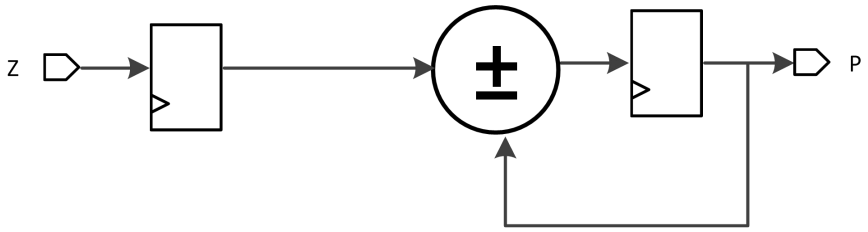

图 5-5 DSP 累加操作简图

使用 DSP 子单元搭建累加器，结构图如图 5-6 所示：

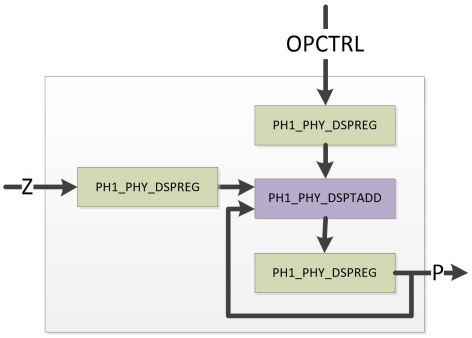

图 5-6 DSP 累加操作结构图

5.4 乘加

DSP 实现乘加功能：P=X*Y+Z

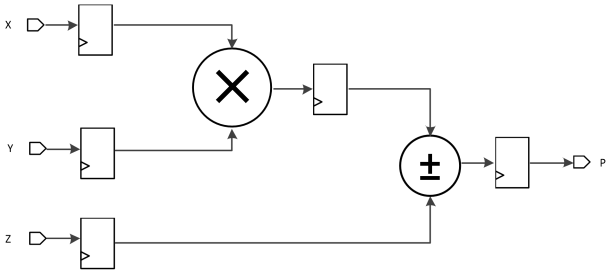

图 5-7 DSP 乘加操作简图

使用 DSP 子单元搭建乘加器，结构图如图 5-8 所示：

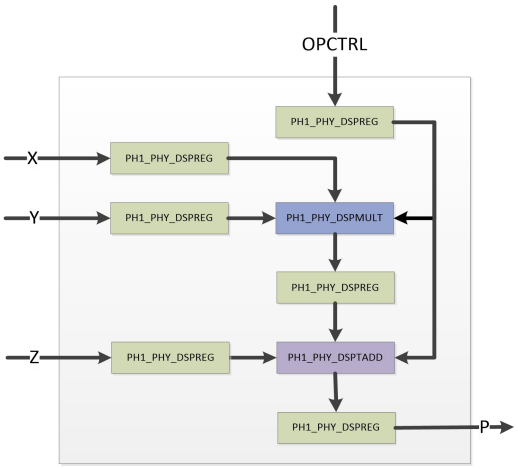

图 5-8 DSP 乘加操作结构图

5.5 乘累加

DSP 实现乘累加功能：P=P±X*Y

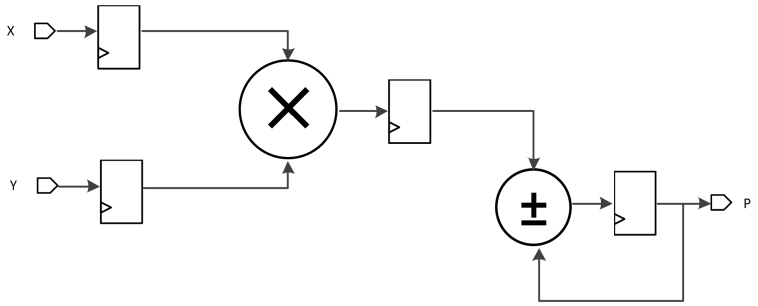

图 5-9 DSP 乘累加操作简图

使用 DSP 子单元搭建乘累加，结构图如图 5-10 所示：

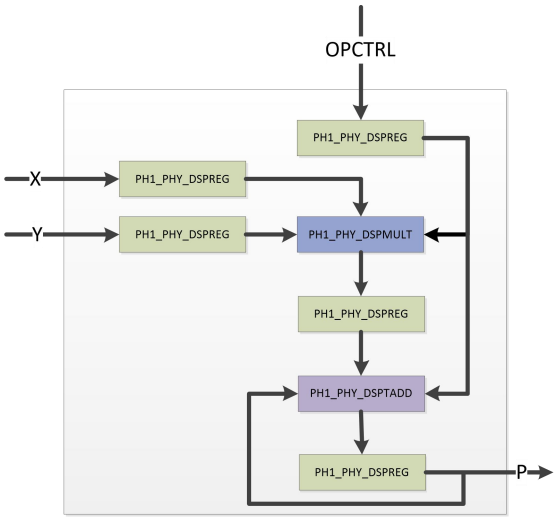

图 5-10 DSP 乘累加操作结构图

5.6 带预加的乘法

DSP 实现带预加的乘法：P=(XA±XB)*Y

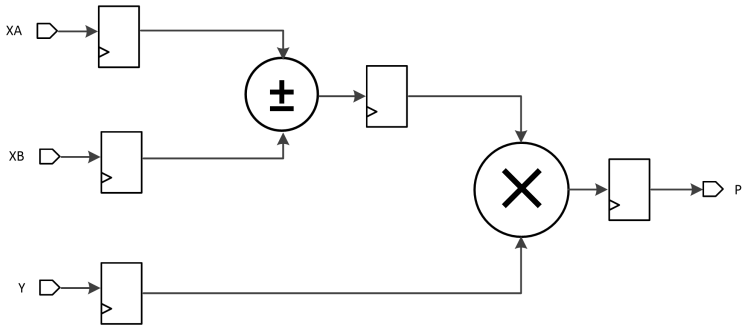

图 5-11 DSP 带预加乘法操作简图

使用 DSP 子单元搭建带预加的乘法，结构图如图 5-12 所示：

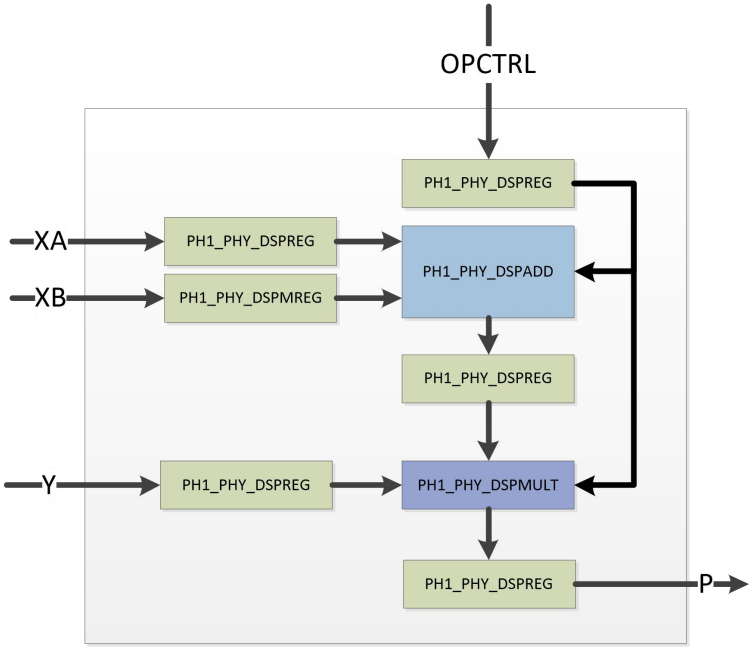

图 5-12 DSP 带预加乘法操作结构图

5.7 带预加的乘累加

DSP 实现带预加的乘累加：P = P ±（XA±XB）*Y

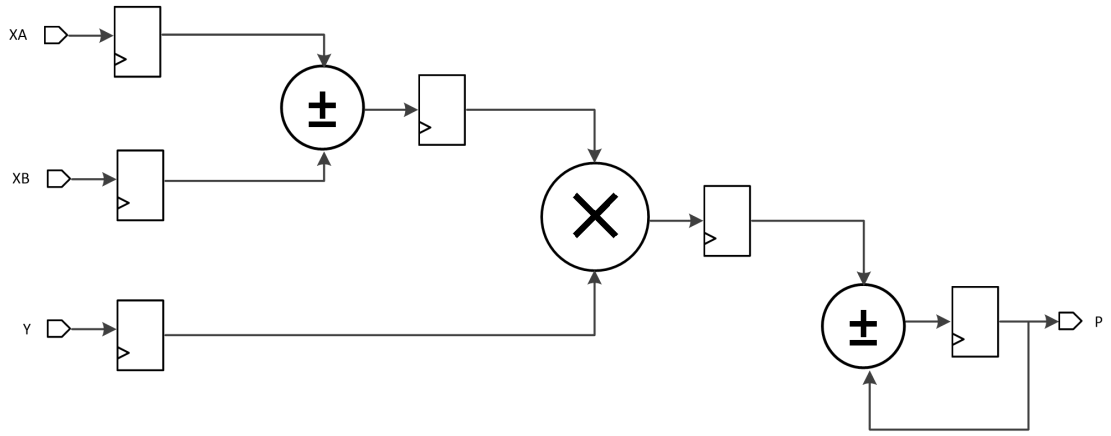

图 5-13 DSP 带预加乘累加操作简图

使用 DSP 子单元搭建带预加的乘累加，结构图如图 5-14 所示：

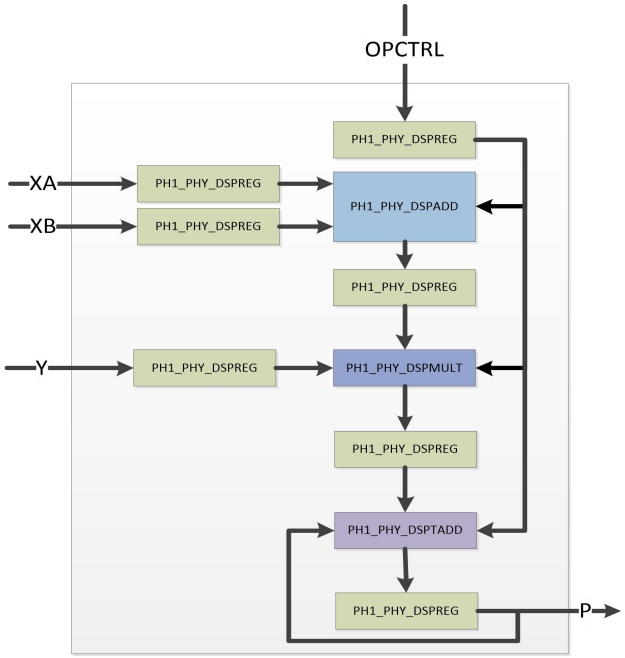

图 5-14 DSP 带预加乘累加操作结构图

5.8 数据动态切换

使用 DSP 计算 P = A*B + C*D，利用 TU 的动态数据切换功能，可进行资源时分复用。

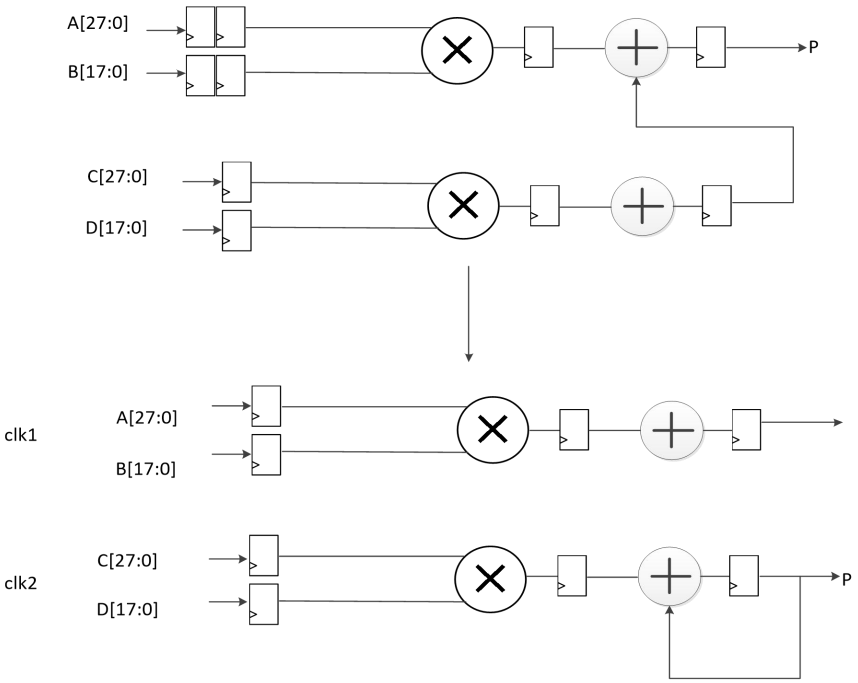

图 5-15 数据动态切换简图

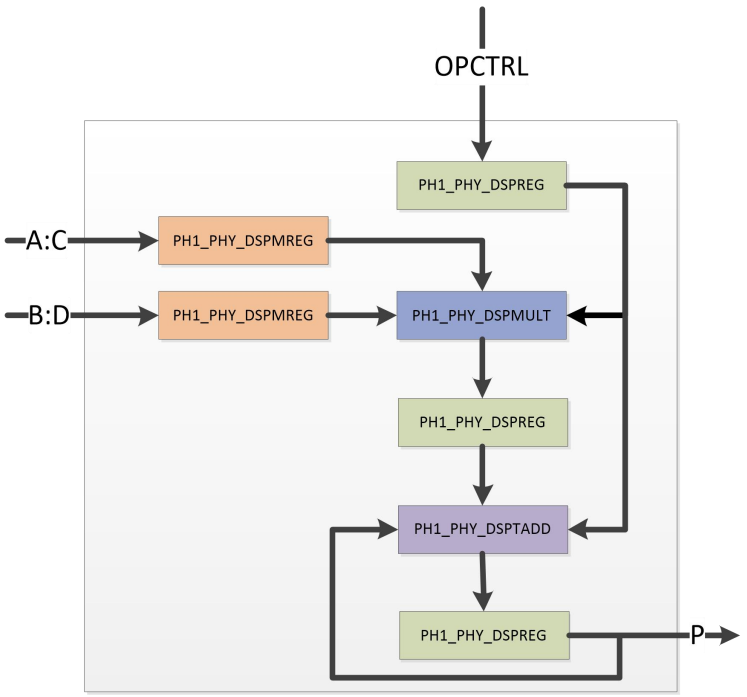

图 5-16 数据动态切换结构图

5.9 级联实现宽位宽加法器

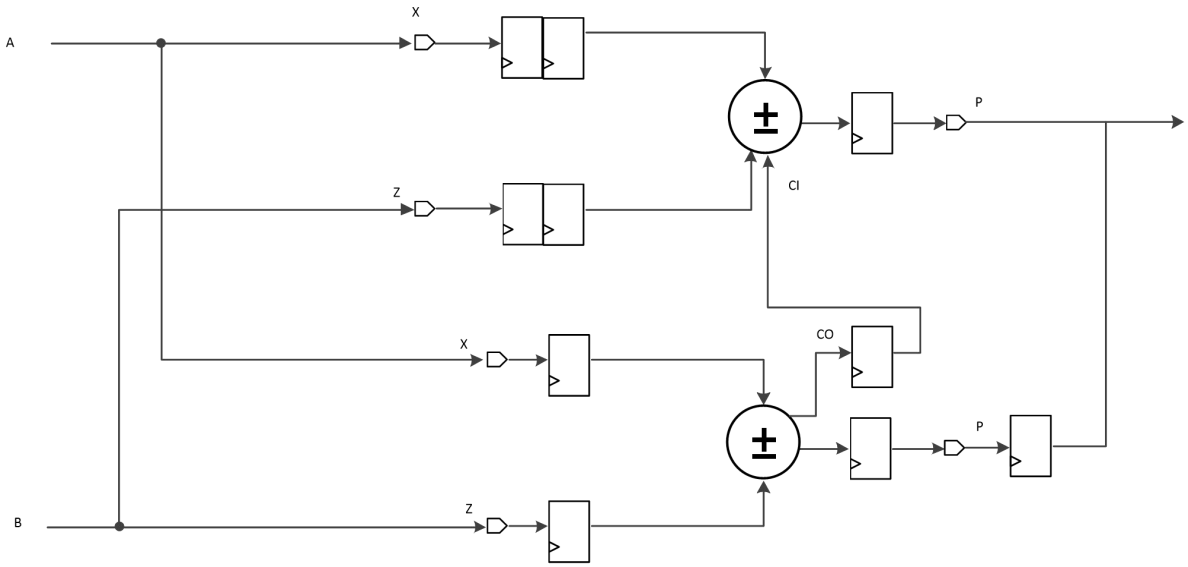

图 5-17 DSP 宽位加法器操作简图

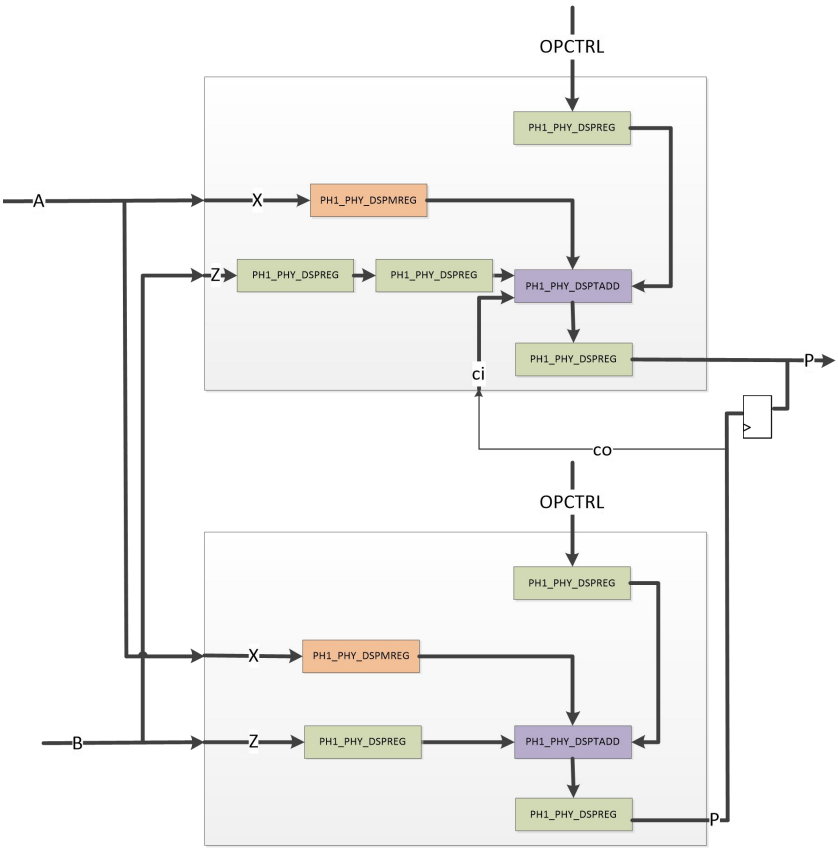

图 5-18 DSP 宽位加法器操作结构图

5.10 级联实现宽位宽乘法器

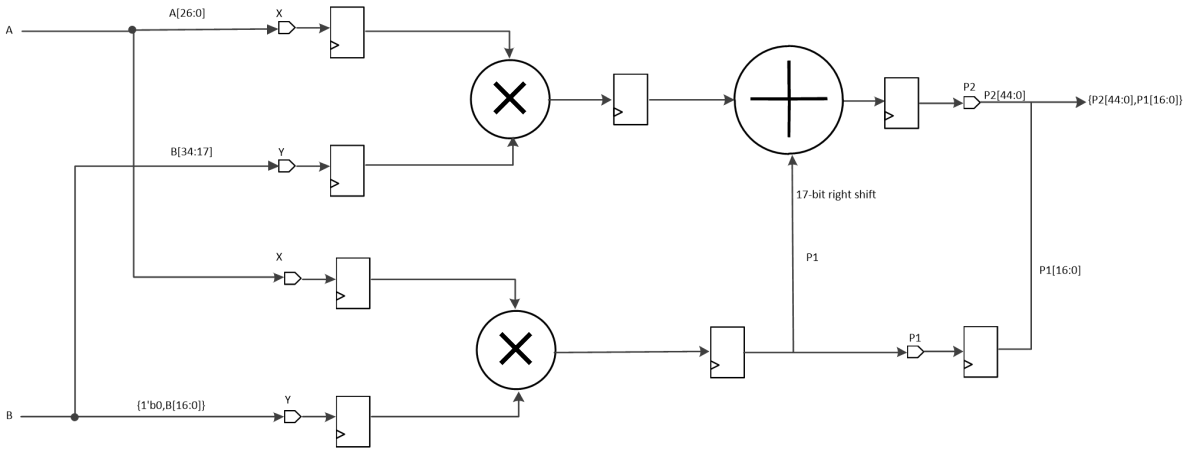

图 5-19 DSP 宽位乘法器操作简图

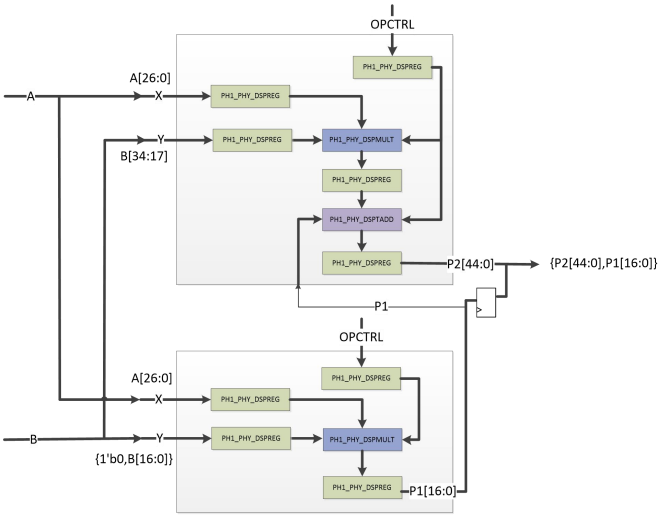

图 5-20 DSP 宽位乘法器操作结构图

5.11 按位逻辑操作

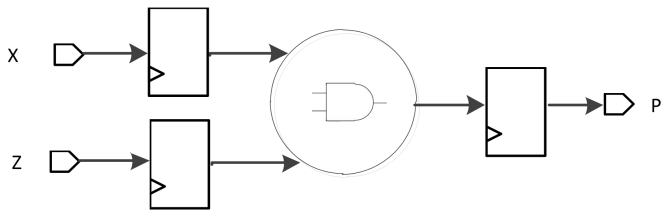

图 5-21 DSP 按位逻辑操作简图

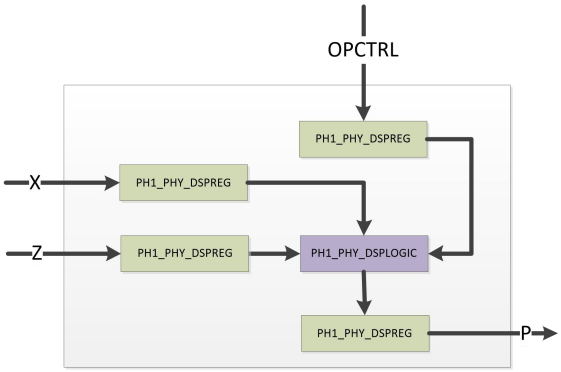

图 5-22 DSP 按位逻辑操作结构图

6 附录 1-组合拼接应用情景示例代码

采用 6 个 dsp primitive 描述的复合计算单元，端口连接后面的注释表示可选连接信号。

```
`timescale 1ns/1ps

module dsp
(
  input wire    clk,
  input wire    rst,
  input wire    ce,

  //输入端口
  input wire [26:0] xa,
  input wire [26:0] xb,
  input wire [17:0] y,
  input wire [53:0] z,
  input wire    ci,

  //控制端口
  input wire [14:0] opctrl,

  //输出端口
  output wire [53:0] p,
  output wire    co,
  output wire    overflow,
  output wire    underflow
);
wire [26:0] xa_in;
wire [26:0] xb_in;
```

```
wire [17:0] y_in;
wire [53:0] z_in;
wire [14:0] opctrl_in;
wire [26:0] sum;
wire [26:0] sum_reg;
wire [44:0] mult;
wire [44:0] mult_reg;
wire [53:0] tsum;
wire [53:0] tsum_reg;
wire tco;
wire toverflow;
wire tunderflow;
wire tco_reg;
wire toverflow_reg;
wire tunderflow_reg;

//xa_in
PH1_PHY_DSPMREG
#(
  .WIDTH     (27      ),
  .RSTMODE    ("ASYNC"    ),
  .CLKMUX     ("SIG"     ),
  .CEMUX     ("1"      ),
  .RSTMUX     ("0"      ),
  .DYNAMIC_DATA  ("Q"      )
)
u_mreg_xa
(
.clk      (clk      ),
  .rst      (rst      ),
  .ce       (ce       ),
  .d        (xa       ),
  .opctrl     (opctrl[0]   ), //opctrl[0], opctrl_in[0]
  .dynamic_q   (xa_in     )
);

//xb_in
PH1_PHY_DSPMREG
#(
  .WIDTH     (27      ),
  .RSTMODE    ("ASYNC"    ),
  .CLKMUX     ("SIG"     ),
  .CEMUX     ("1"      ),
  .RSTMUX     ("0"      ),
```

```
  .DYNAMIC_DATA ("Q"      )
)
u_mreg_xb
(
  .clk      (clk      ),
  .rst      (rst      ),
  .ce       (ce       ),
  .d        (xb       ),
  .opctrl     (opctrl[1]   ), //opctrl[1], opctrl_in[1]
  .dynamic_q   (xb_in     )
);

//y_in
PH1_PHY_DSPMREG
#(
  .WIDTH     (18       ),
  .RSTMODE    ("ASYNC"    ),
  .CLKMUX     ("SIG"      ),
  .CEMUX      ("1"        ),
  .RSTMUX     ("0"        ),
  .DYNAMIC_DATA ("Q"      )
)
u_mreg_y
(
  .clk      (clk      ),
  .rst      (rst      ),
  .ce       (ce       ),
  .d        (y        ),
  .opctrl     (opctrl[2]   ), //opctrl[2], opctrl_in[2]
  .dynamic_q   (y_in      )
);

//z_in
PH1_PHY_DSPREG
#(
  .WIDTH     (54       ),
  .RSTMODE    ("ASYNC"    ),
  .CLKMUX     ("SIG"      ),
  .CEMUX      ("1"        ),
  .RSTMUX     ("0"        )
)
u_reg_z
(
  .clk      (clk      ),
```

```
  .rst      (rst      ),
  .ce       (ce       ),
  .d        (z        ),
  .q        (z_in     )
);

//opctrl_in
PH1_PHY_DSPREG
#(
  .WIDTH     (15       ),
  .RSTMODE   ("ASYNC"   ),
  .CLKMUX    ("SIG"     ),
  .CEMUX     ("1"       ),
  .RSTMUX    ("0"       )
)
u_reg_opctrl
(
  .clk      (clk      ),
  .rst      (rst      ),
  .ce       (ce       ),
  .d        (opctrl    ),
  .q        (opctrl_in  )
);

//add
PH1_PHY_DSPADD u_add
(
     .a        (xa_in     ),
     .b        (xb_in     ),
     .opctrl    ({opctrl[5],opctrl[3]}), //{opctrl[5],opctrl[3]}, {opctrl_in[5],opctrl_in[3]}
     .sum      (sum      )
);

//add_reg
PH1_PHY_DSPREG
#(
  .WIDTH      (27       ),
  .RSTMODE   ("ASYNC"   ),
  .CLKMUX    ("SIG"     ),
  .CEMUX     ("1"       ),
  .RSTMUX    ("0"       )
)
u_add_reg
(
```

```
  .clk      (clk      ),
  .rst      (rst      ),
  .ce       (ce       ),
  .d        (sum      ),
  .q        (sum_reg    )
);

//mult
PH1_PHY_DSPMULT u_mult
(
  .x        (sum      ), //sum, sum_reg, xa_in
  .y        (y_in     ), //sum, sum_reg, xa_in, y_in
.opctrl     (opctrl[5:4]  ), //opctrl[5:4], opctrl_in[5:4]
  .p        (mult     )
);

//mult_reg
PH1_PHY_DSPREG
#(
  .WIDTH     (45      ),
  .RSTMODE    ("ASYNC"    ),
  .CLKMUX     ("SIG"     ),
  .CEMUX      ("1"      ),
  .RSTMUX     ("0"      )
)
u_mult_reg
(
  .clk      (clk      ),
  .rst      (rst      ),
  .ce       (ce       ),
  .d        (mult     ),
  .q        (mult_reg   )
);

//tadd
PH1_PHY_DSPTADD
#(
  .X1_EXTEND   ("NO"     ),
  .Y1_ROUND    ("NO"     ),
  .Z1_SHIFT    ("NO"     ),
  .CI_INVERT   ("NO"     ),
.RND_CONST    (54'b0    ),
  .USE_OVERFLOW  ("S53"     )
)
```

```
u_tadd
(
  .x0       ({xa_in,xb_in} ), // {xa_in,xb_in},0,tsum_reg
  .x1_special    (tsum_reg     ), // {{9{mult[44]}},mult},{{9{mult_reg[44]}},mult_reg},{xa_in,xb_in},0,ts//
um_reg
  .y0       (z_in      ), //z,z_in,0
  .y1_special   (0        ), //z,z_in,0
  .z0       (        ), //tsum_reg,z,z_in,0
  .z1_special   (0        ), //tsum_reg,z,z_in,0
  .ci       (ci       ), //ci,mult[44],mult_reg[44],co,tsum_reg[53],!tsum_reg[53],0
  .ci_special   (mult[44]    ), //ci,mult[44],mult_reg[44],co,tsum_reg[53],!tsum_reg[53],0
  .opctrl     (opctrl_in[14:6]), //opctrl
  .sum      (tsum      ),
  .co       (tco       ),
  .overflow    (toverflow   ),
  .underflow   (tunderflow   )
);

//tadd_reg
PH1_PHY_DSPREG
#(
  .WIDTH      (54       ),
  .RSTMODE     ("ASYNC"     ),
  .CLKMUX      ("SIG"      ),
  .CEMUX      ("1"       ),
  .RSTMUX      ("0"       )
)
u0_tadd_reg
(
  .clk       (clk       ),
  .rst       (rst       ),
  .ce        (ce        ),
  .d        (tsum      ),
  .q        (tsum_reg    )
);

//tadd_reg
PH1_PHY_DSPREG
#(
  .WIDTH      (1        ),
  .RSTMODE     ("ASYNC"     ),
  .CLKMUX      ("SIG"      ),
  .CEMUX      ("1"       ),
  .RSTMUX      ("0"       )
```

```
)
u1_tadd_reg
(
  .clk      (clk      ),
  .rst      (rst      ),
  .ce       (ce       ),
  .d        (tco      ),
  .q        (tco_reg  )
);

//tadd_reg
PH1_PHY_DSPREG
#(
  .WIDTH     (1       ),
  .RSTMODE   ("ASYNC"  ),
  .CLKMUX    ("SIG"    ),
  .CEMUX     ("1"      ),
  .RSTMUX    ("0"      )
)
u2_tadd_reg (
  .clk      (clk      ),
  .rst      (rst      ),
  .ce       (ce       ),
  .d        (toverflow  ),
  .q        (toverflow_reg )
);

//tadd_reg
PH1_PHY_DSPREG
#(
  .WIDTH     (1       ),
  .RSTMODE   ("ASYNC"  ),
  .CLKMUX    ("SIG"    ),
  .CEMUX     ("1"      ),
  .RSTMUX    ("0"      )
)
u3_tadd_reg
(
  .clk      (clk      ),
  .rst      (rst      ),
  .ce       (ce       ),
  .d        (tunderflow  ),
  .q        (tunderflow_reg)
);
```

```
assign p = tsum; // tsum, tsum_reg, mult, mult_reg;
assign overflow = toverflow; // toverflow, toverflow_reg
assign underflow = tunderflow; // toverflow, tunderflow_reg
assign co = tco; // tco, tco_reg
endmodule
```

7 版本信息

日期	版本	修订记录
2021/06/19	0.1	首次发布中文版
2021/10/1	0.2	删除 DSP 模块频率指标描述
2022/01/20	1.0	首次发布正式版本
2022/06/28	1.1	统一修订 PH1A400 系列名称为 PH1A 系列

版权所有©2022 上海安路信息科技股份有限公司

未经本公司书面许可，任何单位和个人都不得擅自摘抄、复制、翻译本文档内容的部分或全部，并不得以任何形式传播。

免责声明

本文档并未授予任何知识产权的许可，并未以明示或暗示，或以禁止发言或其它方式授予任何知识产权许可。除安路科技在其产品的销售条款和条件中声明的责任之外，安路科技概不承担任何法律或非法律责任。安路科技对安路科技产品的销售和/或使用不作任何明示或暗示的担保，包括对产品的特定用途适用性、适销性或对任何专利权、版权或其它知识产权的侵权责任等，均不作担保。安路科技对文档中包含的文字、图片及其它内容的准确性和完整性不承担任何法律或非法律责任，安路科技保留修改文档中任何内容的权利，恕不另行通知。安路科技不承诺对这些文档进行适时的更新。